ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ

Σχολή Ηλεκτρολόγων Μηχανικών & Μηχανικών Η/Υ

Εξάμηνο 8ο: Ψηφιακά VLSI

ΟΜΑΔΑ 23 - ΑΝΑΦΟΡΑ LAB2

Όνομα	Επώνυμο	Α.Μ.
Μιχαήλ	Παναγόπουλος	03122061
Αθανάσιος	Φειδάκης	03122676

➢ Ερώτημα 1 – sync Full Adder

Κώδικας VHDL – Source Code

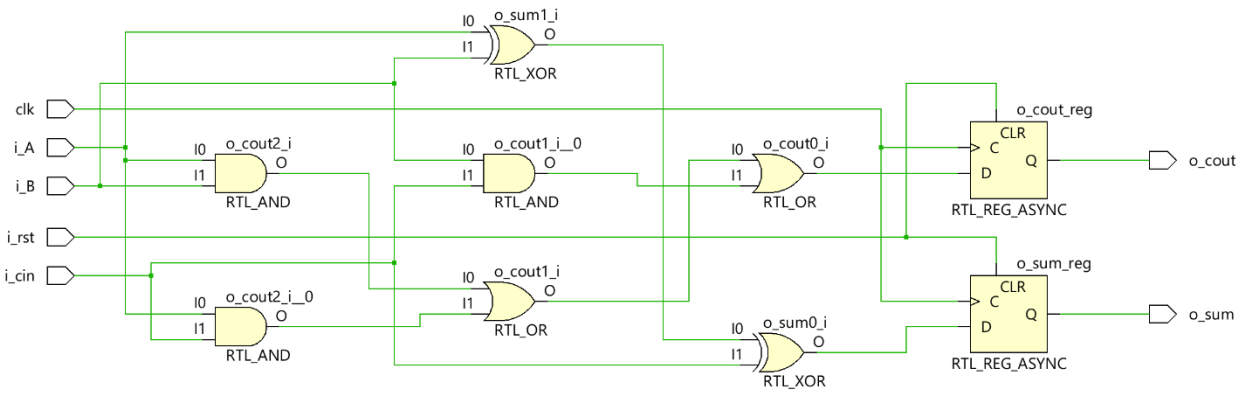
```
library IEEE;
use IEEE.std_logic_1164.all;

-- check for reset
entity sync_FA is
    port (
        i_rst               : in  std_logic;
        clk                 : in  std_logic;
        i_A, i_B, i_cin     : in  std_logic;
        o_sum, o_cout       : out std_logic
    );
end entity;

architecture arc_sync_FA of sync_FA is
begin

    process(clk,i_rst)
    begin
        if i_rst = '1' then
            o_sum   <= '0';
            o_cout  <= '0';
        elsif rising_edge(clk) then
            o_sum   <= i_A xor i_B xor i_cin;
            o_cout  <= (i_A and i_B) or (i_A and i_cin) or (i_B and i_cin);
        end if;
    end process;

end architecture;

```

Κώδικας VHDL – Testbench

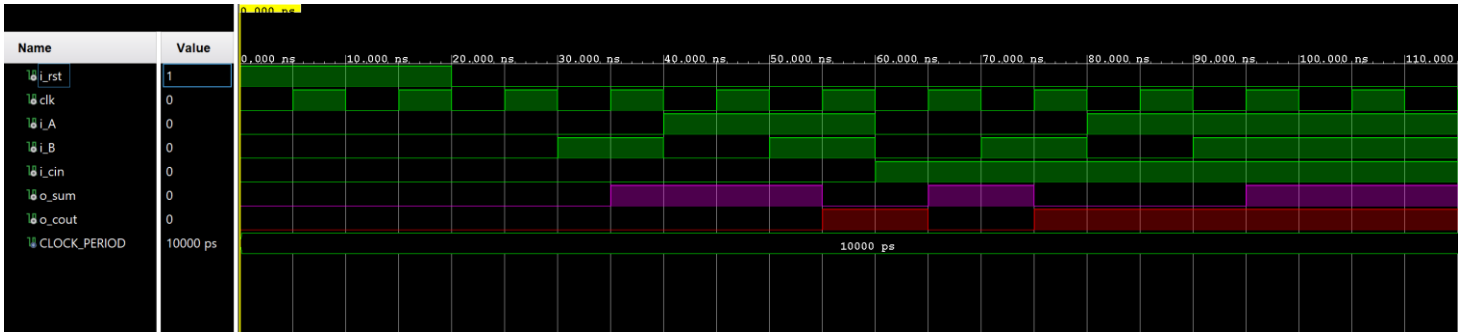
```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity sync_FA_tb is
end entity;

architecture tb of sync_FA_tb is

    component sync_FA is
        port (
            i_rst               : in  std_logic;
            clk                 : in  std_logic;
            i_A, i_B, i_cin     : in  std_logic;
            o_sum, o_cout       : out std_logic
        );
    end component;

    signal i_rst                : std_logic;
    signal clk                  : std_logic;
    signal i_A, i_B, i_cin      : std_logic;
    signal o_sum, o_cout        : std_logic;

    constant CLOCK_PERIOD : time := 10 ns;

begin

```

```
    DUT: sync_FA
        port map (
            i_rst   => i_rst,
            clk     => clk,
            i_A     => i_A,
            i_B     => i_B,
            i_cin   => i_cin,
            o_sum   => o_sum,
            o_cout  => o_cout
        );

    clk_process: process
    begin
        clk <= '0';
        wait for CLOCK_PERIOD/2;
        clk <= '1';
        wait for CLOCK_PERIOD/2;
    end process;

    STIMULUS: process
        variable test_vector : std_logic_vector(3-1 downto 0);
    begin
        i_rst <= '1';
        i_A   <= '0';
        i_B   <= '0';
        i_cin <= '0';
        wait for (2*CLOCK_PERIOD);
        i_rst <= '0';

        for i in 0 to 7 loop
            test_vector := std_logic_vector(to_unsigned(i,3));
            i_cin   <= test_vector(2);
            i_A     <= test_vector(1);
            i_B     <= test_vector(0);
            wait for (1*CLOCK_PERIOD);
        end loop;

        wait;
    end process;

end architecture;

```

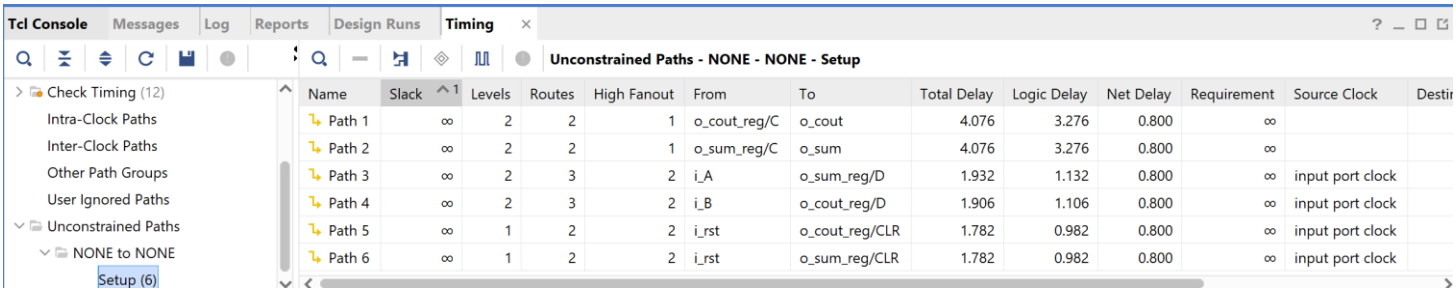
RTL ANALYSIS – Schematic



SIMULATION



Timing Report



Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Source Clock	Destin
Path 1	∞	2	2	1	o_cout_reg/C	o_cout	4.076	3.276	0.800	∞		
Path 2	∞	2	2	1	o_sum_reg/C	o_sum	4.076	3.276	0.800	∞		
Path 3	∞	2	3	2	i_A	o_sum_reg/D	1.932	1.132	0.800	∞	input port clock	
Path 4	∞	2	3	2	i_B	o_cout_reg/D	1.906	1.106	0.800	∞	input port clock	
Path 5	∞	1	2	2	i_rst	o_cout_reg/CLR	1.782	0.982	0.800	∞	input port clock	
Path 6	∞	1	2	2	i_rst	o_sum_reg/CLR	1.782	0.982	0.800	∞	input port clock	

Σύντομη Περιγραφή Υλοποίησης

Ο παρεχόμενος κώδικας VHDL περιγράφει τη σχεδίαση και την επαλήθευση ενός σύγχρονου πλήρους αθροιστή (Synchronous Full Adder), ο οποίος μετατρέπεται από συνδυαστικό σε ακολουθιακό κύκλωμα μέσω της χρήσης ενός process που ενεργοποιείται στη θετική ακμή του ρολογιού (rising edge), ενσωματώνοντας παράλληλα μια ασύγχρονη είσοδο μηδενισμού (active-high reset). Κατά την προσομοίωση του testbench, αναμένουμε οι έξοδοι o_sum και o_cout να ενημερώνονται με καθυστέρηση ενός κύκλου ρολογιού σε σχέση με τις εισόδους i_A, i_B και i_cin, ακολουθώντας πιστά τον πίνακα αλήθειας του πλήρους αθροιστή για όλους τους πιθανούς συνδυασμούς (0-7) μετά την παύση του σήματος reset. Στο RTL σχήμα, η δομή θα απεικονίζεται ως ένας συνδυαστικός αθροιστής του οποίου οι έξοδοι οδηγούνται σε flip-flops, γεγονός που καθορίζει το critical path από την ακμή του ρολογιού έως την έξοδο, περιλαμβάνοντας τον χρόνο διάδοσης των λογικών πυλών XOR και AND/OR συν τον χρόνο αποκατάστασης των καταχωρητών.

➢ Ερώτημα 2 – sync RCA

Κώδικας VHDL – Source Code

```
library IEEE;
use IEEE.std_logic_1164.all;

-- T_latency = (3*CLOCK_PERIOD)

entity sync_RCA4 is
    port (
        i_rst       : in  std_logic;
        clk         : in  std_logic;
        i_A, i_B    : in  std_logic_vector(4-1 downto 0);
        i_cin       : in  std_logic;
        o_sum       : out std_logic_vector(4-1 downto 0);
        o_cout      : out std_logic
    );
end entity;

architecture arc_sync_RCA4 of sync_RCA4 is

    component sync_FA is
        port (
            i_rst               : in  std_logic;
            clk                 : in  std_logic;
            i_A, i_B, i_cin     : in  std_logic;
            o_sum, o_cout       : out std_logic
        );
    end component;

    signal fa_cout              : std_logic_vector(3-1 downto 0);
    signal s_sum                : std_logic_vector(3-1 downto 0);

    signal shift_reg_fa1_sum    : std_logic_vector(3-1 downto 0);

    signal shift_reg_fa2_sum    : std_logic_vector(2-1 downto 0);
    signal shift_reg_fa2_A      : std_logic;
    signal shift_reg_fa2_B      : std_logic;

    signal shift_reg_fa3_sum    : std_logic;
    signal shift_reg_fa3_A      : std_logic_vector(2-1 downto 0);
    signal shift_reg_fa3_B      : std_logic_vector(2-1 downto 0);

    signal shift_reg_fa4_A      : std_logic_vector(3-1 downto 0);
    signal shift_reg_fa4_B      : std_logic_vector(3-1 downto 0);

begin

    fa1 : sync_FA
        port map (
            i_rst   => i_rst,
            clk     => clk,
            i_A     => i_A(0),
            i_B     => i_B(0),
            i_cin   => i_cin,
            o_sum   => s_sum(0),
            o_cout  => fa_cout(0)
        );

    fa2 : sync_FA
        port map (
            i_rst   => i_rst,
            clk     => clk,
            i_A     => shift_reg_fa2_A,
            i_B     => shift_reg_fa2_B,
            i_cin   => fa_cout(0),
```

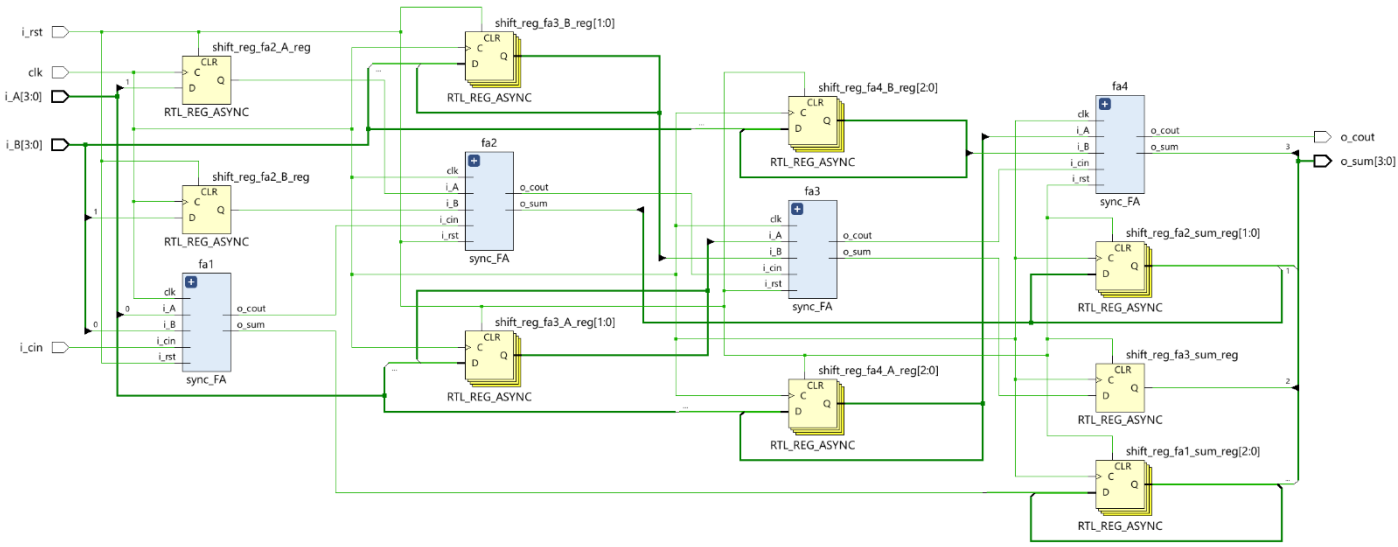
```
            o_sum   => s_sum(1),
            o_cout  => fa_cout(1)
        );

    fa3 : sync_FA
        port map (
            i_rst   => i_rst,
            clk     => clk,
            i_A     => shift_reg_fa3_A(1),
            i_B     => shift_reg_fa3_B(1),
            i_cin   => fa_cout(1),
            o_sum   => s_sum(2),
            o_cout  => fa_cout(2)
        );

    fa4 : sync_FA
        port map (
            i_rst   => i_rst,
            clk     => clk,
            i_A     => shift_reg_fa4_A(2),
            i_B     => shift_reg_fa4_B(2),
            i_cin   => fa_cout(2),
            o_sum   => o_sum(3),
            o_cout  => o_cout
        );

    process(clk,i_rst)
    begin
        if i_rst = '1' then
            --Μηδενισμός καταχωρητών εξόδου
            shift_reg_fa1_sum <= (others => '0');
            shift_reg_fa2_sum <= (others => '0');
            shift_reg_fa3_sum <= '0';

            --Μηδενισμός καταχωρητών εισόδου
            shift_reg_fa2_A   <= '0';
            shift_reg_fa2_B   <= '0';
            shift_reg_fa3_A   <= (others => '0');
            shift_reg_fa3_B   <= (others => '0');
            shift_reg_fa4_A   <= (others => '0');
            shift_reg_fa4_B   <= (others => '0');
        elsif rising_edge(clk) then
            --fa1
            shift_reg_fa1_sum <= shift_reg_fa1_sum(2-1 downto 0) & s_sum(0);

            --fa2
            shift_reg_fa2_A <= i_A(1);
            shift_reg_fa2_B <= i_B(1);

            shift_reg_fa2_sum <= shift_reg_fa2_sum(0) & s_sum(1);

            --fa3
            shift_reg_fa3_A <= shift_reg_fa3_A(0) & i_A(2);
            shift_reg_fa3_B <= shift_reg_fa3_B(0) & i_B(2);

            shift_reg_fa3_sum <= s_sum(2);

            --fa4
            shift_reg_fa4_A <= shift_reg_fa4_A(2-1 downto 0) & i_A(3);
            shift_reg_fa4_B <= shift_reg_fa4_B(2-1 downto 0) & i_B(3);
        end if;
    end process;

    o_sum(0) <= shift_reg_fa1_sum(3-1);
    o_sum(1) <= shift_reg_fa2_sum(1);
    o_sum(2) <= shift_reg_fa3_sum;
end architecture;
```

Κώδικας VHDL – Testbench

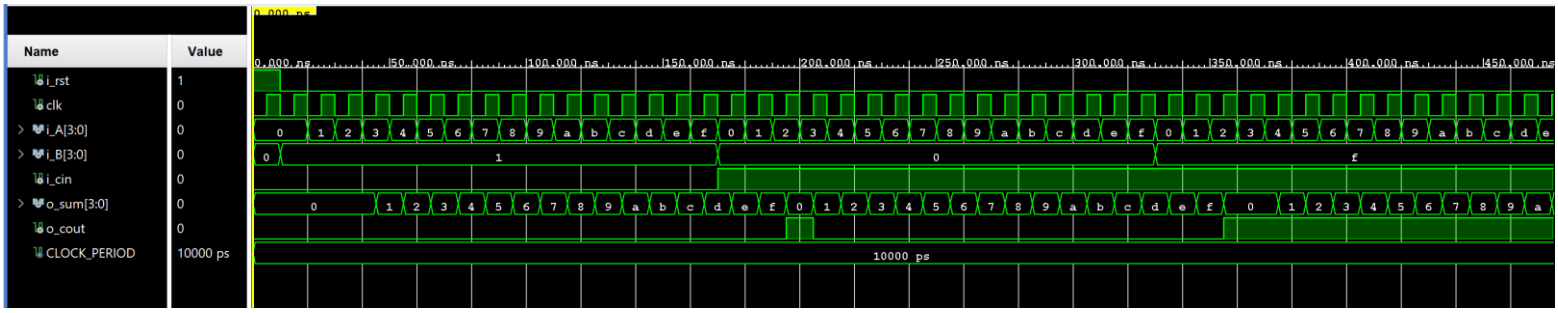
```
 1. library IEEE;
 2. use IEEE.std_logic_1164.all;
 3. use IEEE.numeric_std.all;
 4.
 5. entity sync_RCA_tb is
 6. end entity;
 7.
 8. architecture tb of sync_RCA_tb is
 9.
10.     component sync_RCA4 is
11.         port (
12.             i_rst       : in  std_logic;
13.             clk         : in  std_logic;
14.             i_A, i_B    : in  std_logic_vector(4-1 downto 0);
15.             i_cin       : in  std_logic;
16.             o_sum       : out std_logic_vector(4-1 downto 0);
17.             o_cout      : out std_logic
18.         );
19.     end component;
20.
21.     signal i_rst       : std_logic;
22.     signal clk         : std_logic;
23.     signal i_A, i_B    : std_logic_vector(4-1 downto 0);
24.     signal i_cin       : std_logic;
25.     signal o_sum       : std_logic_vector(4-1 downto 0);
26.     signal o_cout      : std_logic;
27.
28.     constant CLOCK_PERIOD : time := 10 ns; --check this
29.
30. begin
31.
32.     DUT : sync_RCA4
33.         port map (
34.             i_rst   => i_rst,
35.             clk     => clk,
36.             i_A     => i_A,
37.             i_B     => i_B,
38.             i_cin   => i_cin,
39.             o_sum   => o_sum,
40.             o_cout  => o_cout
41.         );
42.
43.     clk_process: process
44.     begin
45.         clk <= '0';
46.         wait for CLOCK_PERIOD/2;
47.         clk <= '1';
48.         wait for CLOCK_PERIOD/2;
49.     end process;
50.
51.     STIMULUS: process
52.     begin
53.
54.         i_rst <= '1';
55.         i_A <= (others => '0');
56.         i_B <= (others => '0');
57.         i_cin <= '0';
58.         wait for (1*CLOCK_PERIOD);
59.         i_rst <=  '0';
60.
61.         i_B <= "0001";
62.         for i in 0 to 15 loop
63.             i_A <= std_logic_vector(to_unsigned(i, 4));
64.             wait for (1*CLOCK_PERIOD);
65.         end loop;
66.
67.         i_rst <= '1';
```

```
        wait for (1*CLOCK_PERIOD);
        i_rst <= '0';

        i_cin <= '1';
        i_B <= (others => '0');
        for i in 0 to 15 loop
            i_A <= std_logic_vector(to_unsigned(i, 4));
            wait for (1*CLOCK_PERIOD);
        end loop;

        wait;
    end process;

end architecture;

```

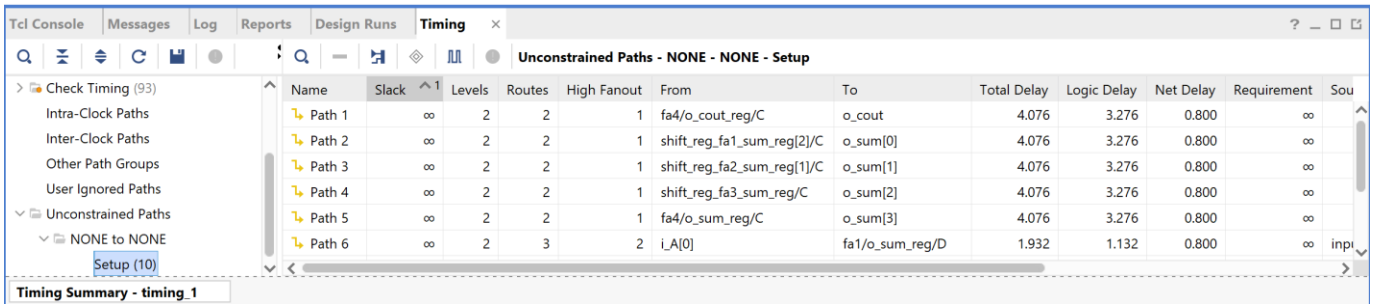
RTL ANALYSIS – Schematic



SIMULATION



Timing Report



Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay	Requirement	Sou
Path 1	∞	2	2	1	fa4/o_cout_reg/C	o_cout	4.076	3.276	0.800	∞	
Path 2	∞	2	2	1	shift_reg_fa1_sum_reg[2]/C	o_sum[0]	4.076	3.276	0.800	∞	
Path 3	∞	2	2	1	shift_reg_fa2_sum_reg[1]/C	o_sum[1]	4.076	3.276	0.800	∞	
Path 4	∞	2	2	1	shift_reg_fa3_sum_reg/C	o_sum[2]	4.076	3.276	0.800	∞	
Path 5	∞	2	2	1	fa4/o_sum_reg/C	o_sum[3]	4.076	3.276	0.800	∞	
Path 6	∞	2	3	2	i_A[0]	fa1/o_sum_reg/D	1.932	1.132	0.800	∞	inpu

Σύντομη Περιγραφή Υλοποίησης

Ο κώδικας περιγράφει έναν σύγχρονο αθροιστή σειριακής μεταφοράς κρατουμένου 4-bit (4-bit Pipelined Ripple Carry Adder), ο οποίος υλοποιεί μια πλήρη δομή pipeline για τη μεγιστοποίηση της συχνότητας λειτουργίας. Σε αντίθεση με έναν απλό συνδυαστικό αθροιστή όπου το κρατούμενο πρέπει να διαδοθεί μέσα από όλα τα bits σε έναν κύκλο, εδώ η σχεδίαση χρησιμοποιεί ενδιάμεσους καταχωρητές (shift registers) για τον συγχρονισμό των εισόδων και των αποτελεσμάτων, έτσι ώστε κάθε στάδιο Full Adder να επεξεργάζεται τα δεδομένα του στον δικό του κύκλο ρολογιού. Κατά την προσομοίωση του testbench, αναμένουμε μια καθυστέρηση (latency) 4 κύκλων ρολογιού μέχρι να εμφανιστεί το πρώτο έγκυρο αποτέλεσμα στην έξοδο, καθώς τα δεδομένα πρέπει να περάσουν από τα διαδοχικά στάδια των καταχωρητών ολίσθησης που έχουν οριστεί για τα σήματα i_A, i_B και s_sum. Στο RTL σχήμα, θα εμφανιστεί μια κλιμακωτή δομή από Flip-Flops που περιβάλλουν κάθε Full Adder, εξασφαλίζοντας ότι το critical path περιορίζεται πλέον στη διαδρομή μέσα σε έναν μόνο Full Adder (από το ρολόι ενός καταχωρητή έως την είσοδο του επόμενου) και όχι στη συνολική διαδρομή και των 4 bits. Αυτή η προσέγγιση pipeline προσφέρει το σημαντικό πλεονέκτημα της δραστικής μείωσης της κρίσιμης διαδρομής σε σύγκριση με έναν κλασικό συνδυαστικό αθροιστή 4-bit, επιτρέποντας στο κύκλωμα να λειτουργεί σε πολύ υψηλότερες συχνότητες ρολογιού, θυσιάζοντας λίγους κύκλους αρχικής καθυστέρησης για χάρη του αυξημένου throughput.

➢ Ερώτημα 3 – sync RCM

Κώδικας VHDL – Source Code

mul_cell

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity mul_cell is
    port (
        clk     : in  std_logic;
        i_rst   : in  std_logic;
        i_a     : in  std_logic;
        i_b     : in  std_logic;
        i_s     : in  std_logic;
        i_c     : in  std_logic;
        o_a     : out std_logic;
        o_b     : out std_logic;
        o_s     : out std_logic;
        o_c     : out std_logic
    );
end entity;

architecture arc_mul_cell of mul_cell is

    component sync_FA is
        port (
            i_rst               : in  std_logic;
            clk                 : in  std_logic;
            i_A, i_B, i_cin     : in  std_logic;
            o_sum, o_cout       : out std_logic
        );
    end component;

    signal ab       : std_logic;
    signal a_temp   : std_logic;

begin

    sFA1: sync_FA
        port map (
            i_rst   => i_rst,
            clk     => clk,
            i_A     => ab,
            i_B     => i_s,
            i_cin   => i_c,
            o_sum   => o_s,
            o_cout  => o_c
        );

    process(clk, i_rst)
    begin
        if i_rst = '1' then
            a_temp  <= '0';
            o_a     <= '0';
            o_b     <= '0';
        elsif rising_edge(clk) then
            o_a     <= a_temp;
            a_temp  <= i_a;
            o_b     <= i_b;
        end if;
    end process;

```

```
    ab  <= i_a and i_b;
end architecture;

```

Multiplier 4

```
   library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

entity sync_RCM4 is
    port (
        clk     : in  std_logic;
        i_rst   : in  std_logic;
        i_A     : in  std_logic_vector(4-1 downto 0);
        i_B     : in  std_logic_vector(4-1 downto 0);
        o_P     : out std_logic_vector(4+4-1 downto 0)
    );
end entity;

architecture arc_sync_RCM4 of sync_RCM4 is

    type a_word_array is array (0 to 4-1) of std_logic_vector(4-1 downto 0);
    signal ai, bi, si, ci : a_word_array;
    signal ao, bo, so, co : a_word_array;

    signal s_reg_p0 : std_logic_vector(9-1 downto 0);
    signal s_reg_p1 : std_logic_vector(7-1 downto 0);
    signal s_reg_p2 : std_logic_vector(5-1 downto 0);
    signal s_reg_p3 : std_logic_vector(3-1 downto 0);
    signal s_reg_p4 : std_logic_vector(2-1 downto 0);
    signal s_reg_p5 : std_logic;

    signal s_reg_a1 : std_logic;
    signal s_reg_a2 : std_logic_vector(1 downto 0);
    signal s_reg_a3 : std_logic_vector(2 downto 0);

    signal s_reg_b1 : std_logic_vector(1 downto 0);
    signal s_reg_b2 : std_logic_vector(3 downto 0);
    signal s_reg_b3 : std_logic_vector(5 downto 0);

    component mul_cell is
        port (
            clk     : in  std_logic;
            i_rst   : in  std_logic;
            i_a     : in  std_logic;
            i_b     : in  std_logic;
            i_s     : in  std_logic;
            i_c     : in  std_logic;
            o_a     : out std_logic;
            o_b     : out std_logic;
            o_s     : out std_logic;
            o_c     : out std_logic
        );
    end component;

begin

    --generation of mul cells
    gcb: for i in 0 to 4-1 generate
        gca: for j in 0 to 4-1 generate
            gc: mul_cell
                port map (
                    clk     => clk,
                    i_rst   => i_rst,
                    i_a     => ai(i)(j),
                    i_b     => bi(i)(j),
```

```
                    i_s     => si(i)(j),
                    i_c     => ci(i)(j),
                    o_a     => ao(i)(j),
                    o_b     => bo(i)(j),
                    o_s     => so(i)(j),
                    o_c     => co(i)(j)
                );
        end generate;
    end generate;

    --generation of intermediate wires
    gcaw: for i in 1 to 4-1 generate
            ai(i)   <= ao(i-1);
            si(i)   <= co(i-1)(4-1) & so(i-1)(4-1 downto 1);
    end generate;

    gcbiw:   for i in 0 to 4-1 generate
        gcbjw:  for j in 1 to 4-1 generate
                bi(i)(j) <= bo(i)(j-1);
                ci(i)(j) <= co(i)(j-1);
        end generate;
    end generate;

    --initialization of signals
    si(0) <= (others => '0');
    gbci: for i in 0 to 4-1 generate
            ci(i)(0) <= '0';
    end generate;

    process(clk,i_rst)
    begin

        if i_rst = '1' then
            s_reg_p0 <= (others => '0');
            s_reg_p1 <= (others => '0');
            s_reg_p2 <= (others => '0');
            s_reg_p3 <= (others => '0');
            s_reg_p4 <= (others => '0');
            s_reg_p5 <= '0';

            s_reg_a1 <= '0';
            s_reg_a2 <= (others => '0');
            s_reg_a3 <= (others => '0');

            s_reg_b1 <= (others => '0');
            s_reg_b2 <= (others => '0');
            s_reg_b3 <= (others => '0');
        elsif rising_edge(clk) then
            --output shift
            s_reg_p0 <= s_reg_p0(8-1 downto 0) & so(0)(0);
            s_reg_p1 <= s_reg_p1(6-1 downto 0) & so(1)(0);
            s_reg_p2 <= s_reg_p2(4-1 downto 0) & so(2)(0);
            s_reg_p3 <= s_reg_p3(2-1 downto 0) & so(3)(0);
            s_reg_p4 <= s_reg_p4(0) & so(3)(1);
            s_reg_p5 <= so(3)(2);

            --input shift
            s_reg_a1 <= i_A(1);
            s_reg_a2 <= s_reg_a2(0) & i_A(2);
            s_reg_a3 <= s_reg_a3(1 downto 0) & i_A(3);

            s_reg_b1 <= s_reg_b1(0) & i_B(1);
            s_reg_b2 <= s_reg_b2(2 downto 0) & i_B(2);
            s_reg_b3 <= s_reg_b3(4 downto 0) & i_B(3);
        end if;
    end process;

    o_P(0) <= s_reg_p0(8);
    o_P(1) <= s_reg_p1(6);
    o_P(2) <= s_reg_p2(4);
```

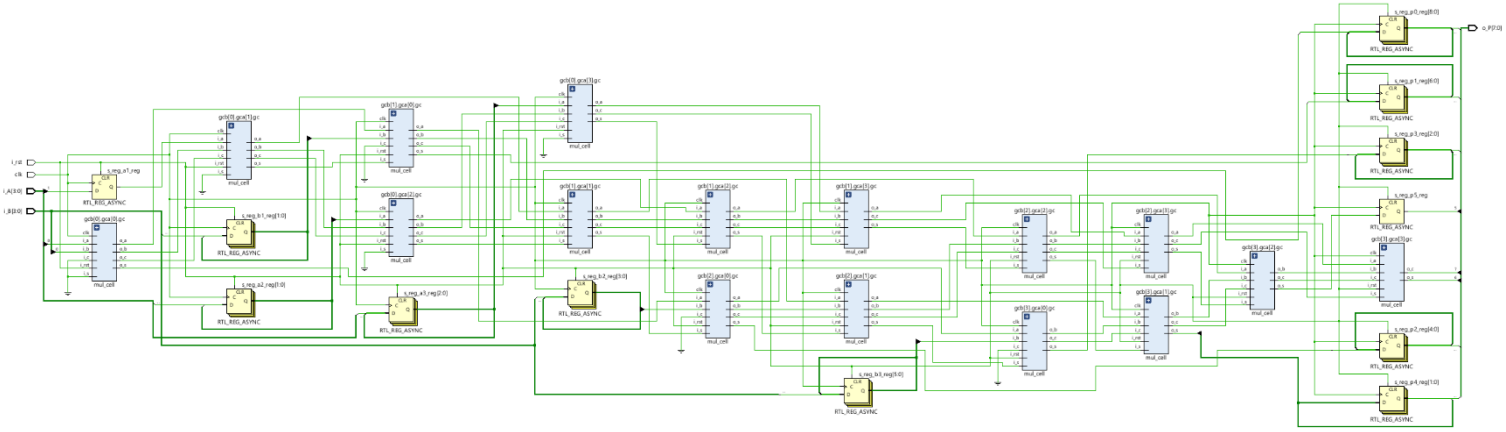
```
    o_P(3) <= s_reg_p3(2);
    o_P(4) <= s_reg_p4(1);
    o_P(5) <= s_reg_p5;
    o_P(6) <= so(3)(3);
    o_P(7) <= co(3)(3);

    ai(0)(0) <= i_A(0);
    ai(0)(1) <= s_reg_a1;
    ai(0)(2) <= s_reg_a2(1);
    ai(0)(3) <= s_reg_a3(2);

    bi(0)(0) <= i_B(0);
    bi(1)(0) <= s_reg_b1(1);
    bi(2)(0) <= s_reg_b2(3);
    bi(3)(0) <= s_reg_b3(5);

end architecture;

```

Κώδικας VHDL – Testbench

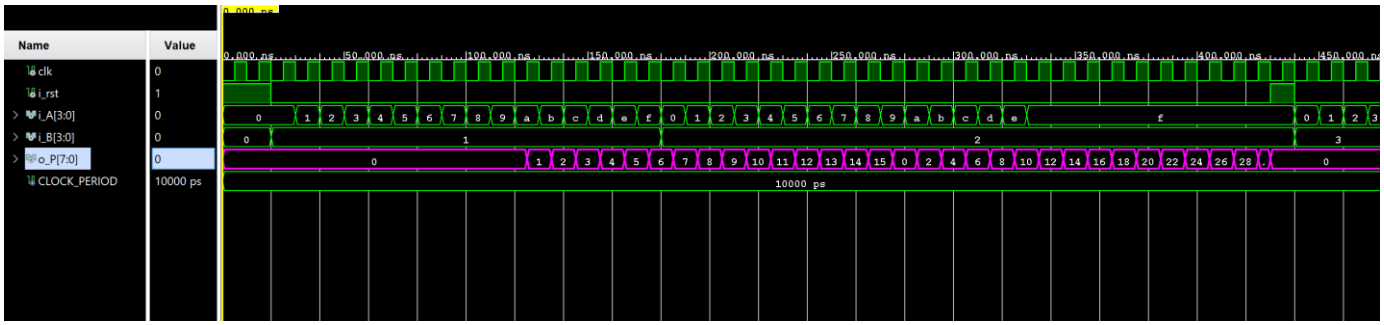
```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.numeric_std.all;

-- Η είσοδος περνάει από 9 registers (καθυστέρησης) για να φθάσει στην έξοδο
-- T_latency: Από ακμή ανάγνωσης input έως ακμή εμφάνισης output
-- T_latency = 9*CLOCK_PERIOD.

entity sync_RCM4_tb is
end entity;

architecture tb of sync_RCM4_tb is

    component sync_RCM4 is
        port (
            clk     : in  std_logic;
            i_rst   : in  std_logic;
            i_A     : in  std_logic_vector(4-1 downto 0);
            i_B     : in  std_logic_vector(4-1 downto 0);
            o_P     : out std_logic_vector(4+4-1 downto 0)
        );
    end component;

    signal clk     : std_logic;
    signal i_rst   : std_logic;
    signal i_A     : std_logic_vector(4-1 downto 0);
    signal i_B     : std_logic_vector(4-1 downto 0);
    signal o_P     : std_logic_vector(4+4-1 downto 0);

    constant CLOCK_PERIOD : time := 10 ns;

begin

    DUT: sync_RCM4
        port map (
            clk     => clk,
            i_rst   => i_rst,
            i_A     => i_A,
            i_B     => i_B,
            o_P     => o_P
        );

    clk_process: process
    begin
```

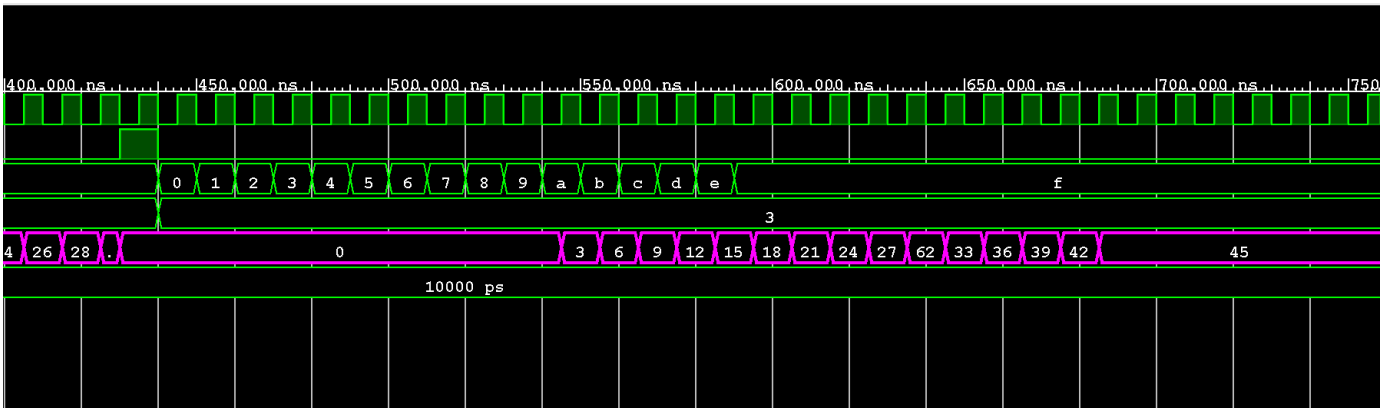
```
        clk <= '0';
        wait for CLOCK_PERIOD/2;
        clk <= '1';
        wait for CLOCK_PERIOD/2;
    end process;

    STIMULUS: process
    begin

        i_rst   <= '1';
        i_A     <= (others => '0');
        i_B     <= (others => '0');
        wait for (2*CLOCK_PERIOD);
        i_rst <= '0';

        i_B <= "0001";
        for i in 0 to 15 loop
            i_A <= std_logic_vector(to_unsigned(i,4));
            wait for (1*CLOCK_PERIOD);
        end loop;

        wait for (9*CLOCK_PERIOD);

        i_rst <= '1';
        wait for (2*CLOCK_PERIOD);
        i_rst <= '0';

        i_B <= "0010";
        for i in 0 to 15 loop
            i_A <= std_logic_vector(to_unsigned(i,4));
            wait for (1*CLOCK_PERIOD);
        end loop;

        wait for (9*CLOCK_PERIOD);

        i_rst <= '1';
        wait for (1*CLOCK_PERIOD);
        i_rst <= '0';

        i_B <= "0011";
        for i in 0 to 15 loop
            i_A <= std_logic_vector(to_unsigned(i,4));
            wait for (1*CLOCK_PERIOD);
        end loop;

        wait;

    end process;

end architecture;

```

RTL ANALYSIS – Schematic

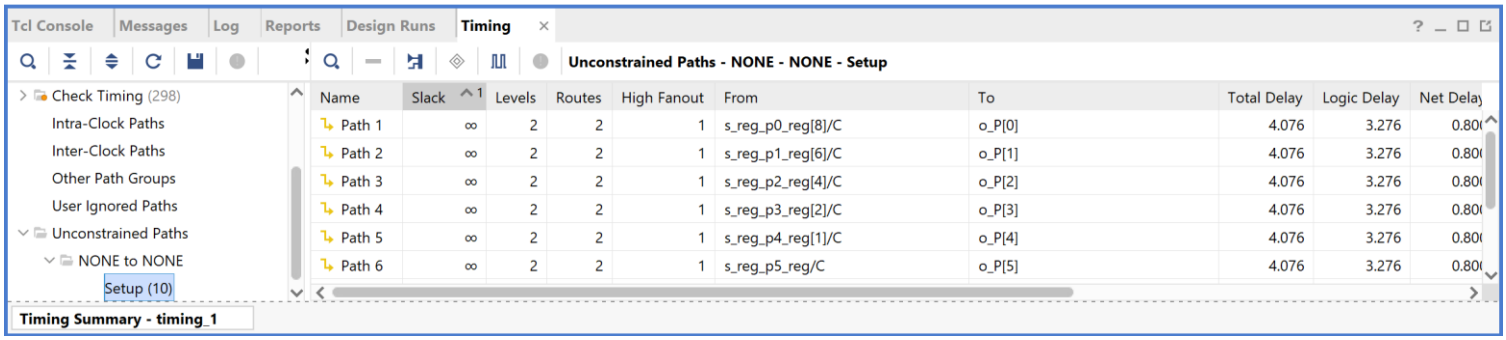


SIMULATION





Timing Report



Name	Slack	Levels	Routes	High Fanout	From	To	Total Delay	Logic Delay	Net Delay
Path 1	∞	2	2	1	s_reg_p0_reg[8]/C	o_P[0]	4.076	3.276	0.80
Path 2	∞	2	2	1	s_reg_p1_reg[6]/C	o_P[1]	4.076	3.276	0.80
Path 3	∞	2	2	1	s_reg_p2_reg[4]/C	o_P[2]	4.076	3.276	0.80
Path 4	∞	2	2	1	s_reg_p3_reg[2]/C	o_P[3]	4.076	3.276	0.80
Path 5	∞	2	2	1	s_reg_p4_reg[1]/C	o_P[4]	4.076	3.276	0.80
Path 6	∞	2	2	1	s_reg_p5_reg/C	o_P[5]	4.076	3.276	0.80

Σύντομη Περιγραφή Υλοποίησης

Ο κώδικας περιγράφει έναν σύγχρονο πολλαπλασιαστή 4-bit τύπου Array Multiplier με πλήρη δομή pipeline, ο οποίος βασίζεται στην επαναλαμβανόμενη χρήση μιας βασικής κυψέλης (mul_cell). Η κυψέλη αυτή αποτελεί τη δομική μονάδα του κυκλώματος, ενσωματώνοντας έναν σύγχρονο πλήρη αθροιστή (sync_FA) για τον υπολογισμό του μερικού γινομένου (ai*bj) και την πρόσθεσή του με το τρέχον άθροισμα και κρατούμενο, ενώ περιλαμβάνει εσωτερικούς καταχωρητές που καθυστερούν τα σήματα εισόδου και εξόδου ώστε κάθε πράξη να ολοκληρώνεται σε έναν κύκλο ρολογιού. Η συνολική αρχιτεκτονική (sync_RCM4) οργανώνει αυτές τις κυψέλες σε μια δισδιάστατη διάταξη 4x4, όπου τα σήματα a, b, s, c διαδίδονται στο πλέγμα με τον κατάλληλο τρόπο. Λόγω της εκτενούς χρήσης pipeline (τόσο εντός των κυψελών όσο και στους εξωτερικούς shift registers ευθυγράμμισης), το critical path περιορίζεται στη λογική μίας μόνο κυψέλης, γεγονός που επιτρέπει εξαιρετικά υψηλές συχνότητες λειτουργίας σε σχέση με έναν συνδυαστικό πολλαπλασιαστή, όπου η καθυστέρηση θα ήταν το άθροισμα όλων των επιμέρους βαθμίδων. Κατά την προσομοίωση, αναμένουμε μια συνολική καθυστέρηση (latency) 9 κύκλων ρολογιού από τη στιγμή που θα εισαχθούν οι τελεστέοι μέχρι την εμφάνιση του πλήρους αποτελέσματος 8-bit στην έξοδο o_P, όπως υποδεικνύεται και στο testbench. Το RTL σχήμα θα αναδεικνύει μια συμμετρική κυψελωτή δομή που περιβάλλεται από συστοιχίες καταχωρητών ολίσθησης (input/output alignment

registers), οι οποίοι είναι απαραίτητοι για να διασφαλιστεί ότι τα bits διαφορετικής βαρύτητας συναντώνται στις σωστές κυψέλες την κατάλληλη χρονική στιγμή, μεγιστοποιώντας έτσι το throughput του συστήματος.